Project: Solving non-Linear PDE for Arolla glacier using FO Model

Devendra Singh

Table of contents

Introduction . . . 2
1. Order of equation : . . . 3
2. Type of equation : . . . 3
3. FEM formalism : . . . 3
4. Linear version of Equation 1 . . . 4
5. Galerkin Orthogonolity . . . 4
6. Best Approximation Result . . . 5
7.Priori estimate for Equation 1 . . . 6
8. Solving the non-linearity . . . 7
9. ϵ and convergence . . . 7
10. Result . . . 8
11. Script used . . . 8

List of Figures

1 Glacial horizontal velocity in m/year (2D) for the constant viscosity case, using $\epsilon = 10^{-15}$. . . . 4
2 Glacial horizontal velocity (km/yr) solved as a non-linear PDE via Picard iteration. Parameters: $\rho = 917$, $A = 3.17 \times 10^{-24}$. . . . 7

List of Tables

1 Physical Model Parameters (SI) . . . 4
2 Convergence in the Picard iteration method with the range of epsilon value . . 7

Introduction

In this project we are going to simulate the Swiss Mountain glacier, Arolla Glacier. In order to do that we are going to model the ice velocity in a 2D cross section represented by the PDE called First order model. These equations are non-linear as the viscosity depends on horizontal velocity(Equation 5) making it an non- Newtonianmesh as dmesh fluid, represented as follows:

FO model:

$$2\partial_x(\eta\partial_x u_x) + \frac{1}{2}\partial_z(\eta\partial_z u_x) = \rho g \partial_x h \tag{1}$$

$$\partial_x u_z = -\partial_x u_x \tag{2}$$

$$\partial_t h + u_x|_{z=s}\partial_x h = u_z \tag{3}$$

Boundary Conditions:

At the bed surface(Dirichlet's): $u_x = 0$

At the ice/atmosphere interface(Neumann):

$$\eta(2\partial_x u_x \partial_x h - \frac{1}{2}\partial_z u_x) = 0 \tag{4}$$

Viscosity dependence on horizontal velocity:

$$\eta = A^{-1/3}\left((\partial_x u_x)^2 + \frac{1}{4}(\partial_z u_x)^2\right)^{-1/3} \tag{5}$$

1. Order of equation :

The Equation 1, is **second order** for horizontal velocity.

2. Type of equation :

For constant viscosity(η), the PDE for horizontal velocity Equation 1 can be written as,

$$\eta(2\partial_{xx} + \frac{1}{2}\partial_{zz})u_x = f \tag{6}$$

When we put values in the discriminat ($B^2 - 4AC$), we get it as negative since the term representing the coefficient B is absent and A, C both are positive, suggesting that its an **elliptic** PDE of second order.

3. FEM formalism :

To solve Equation 1 using FEM, we first need to convert this PDE into weak(Variational) form. For that, we define a function space V over our mesh Ω. In this function space exist our solution $u \in V := H_o^1(\Omega)$ to our continuous variational problem. Therefore to find u,

$$a(u, v) = L(v) \;\forall v \in V$$

the bilinear form

$$a(u, v) = \int_\Omega \left(2\eta\partial_x u\partial_x v + \frac{1}{2}\eta\partial_z u\partial_z v\right), d\Omega \tag{7}$$

the linear form,

$$L(v) = -\int_\Omega \rho g\partial_x hv \, d\Omega \tag{8}$$

and the boundary condition:

At the bed surface(Γ_{bed}): $u_x = 0$

At the ice/atmosphere interface($\Gamma_{surface}$): $\eta(2\partial_x v_x * \partial_x h - \frac{1}{2}\partial_z v_x) = 0$

The set of equations Equation 7, Equation 8, Equation 4 represent the well posed and complete problem which can be solved linearly in FeniCS.

Table 1: Physical Model Parameters (SI)

Variable	Symbol	Value	Units
Ice Density	rho	917.00	kg/m^3
Gravity	g	9.81	m/s^2
Rate Factor	A	3.17×10^{-24}	Pa^-3 s^-1
Linear Viscosity	Constant eta	1.00×10^{15}	Pa s

4. Linear version of Equation 1

To achieve linearity, we have set the viscosity as constant.

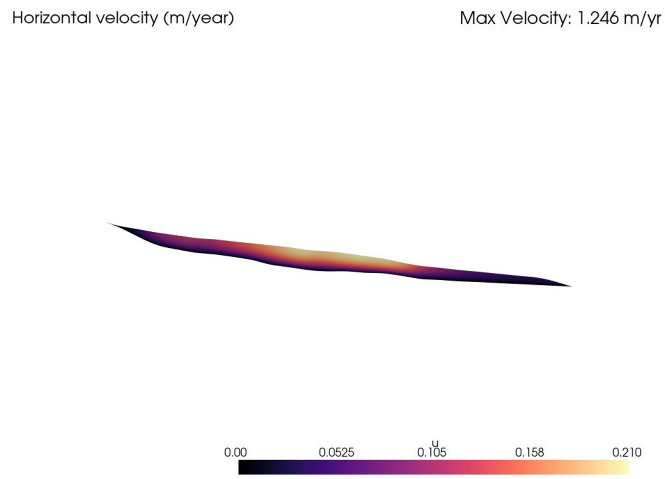

Figure 1: Glacial horizontal velocity in m/year (2D) for the constant viscosity case, using $\epsilon = 10^{-15}$.

5. Galerkin Orthogonolity

The Equation 7 and Equation 8, when solved for Exact Solution $u \in V$ in the combined form can be written as:

$$\int_{\Omega} \left(2\eta \partial_x u \partial_x v + \frac{1}{2} \eta \partial_z u \partial_z v \right), d\Omega = -\int \Omega \rho g \partial_x h v \, d\Omega \;\; \forall v \in V \tag{9}$$

Now we define a finite element space, $V_h = \{v : v \in V, v|_{\Gamma_{bed}} = 0\}$ and find a Discrrete solution $u_h \in V_h$ such that,

$$\int_{\Omega} \left(2\eta\partial_x u_h \partial_x v_h + \frac{1}{2}\eta\partial_z u_h \partial * z v_h * \right), d\Omega = -\int \Omega\rho g\partial_x h v_h, d\Omega \quad \forall v_h \in V_h \tag{10}$$

Since, $V_h \subset V$, we can subtract Equation 10 from Equation 9(within the assumption that ∂_x is similar for both continuous and discrete settings and by the linearity of the integral).The result would be:

$$\int_{\Omega} 2\eta\partial_x(u-u_h)\partial_x v_h + \int_{\Omega} \frac{1}{2}\eta\partial_z(u-u_h)\partial_z v_h \, d\Omega = 0 \ \forall v_h \in V_h \tag{11}$$

This is the Galerkin orthogonality relation suggesting that errors are orthogonal to the solution or in other words the projection of residual errors is zero onto finite element space.

6. Best Approximation Result

For constant viscosity, we need to prove the Best Approximation Result for Equation 1 is the

$$2\eta||\partial_x(u-u_h)||_{L^2(\Omega)} + \frac{1}{2}\eta||\partial_z(u-u_h)||_{L^2(\Omega)}$$

Starting with the associated energy norm: $||v||_a^2 = a(v,v) = \int_{\Omega} 2\eta\partial_x v.\partial_x v dx$

We take an arbitrary $v_h \in V_h$ and analyse the error,

$||u-u_h||_a^2 = a(u-u_h, u-u_h)$

Adding and subtracting a term v_h, $||u-u_h||_a^2 = a(u-u_h, (u-v_h)+(v_h-u_h))$

Expanding we get, $||u-u_h||_a^2 = a(u-u_h, u-v_h) + a(u-u_h, v_h-u_h)$,

since error is orthogonal to the finite element space, the second term vanishes as $v_h - u_h \in V_h$. Now using Cauchy,-Swartz inequality, $a(v,w) \leq C||v||_a||w||_a$ Therefore, $||u-v_h||_a^2 \leq C||u-u_h||_a \ ||u-v_h||_a$ or,

$$||u-u_h||_a \leq C \ ||u-v_h||_a \tag{12}$$

This is our best approximation result.

To calculate M: continuity constant, we use Equation 7,

$$a(u,v) = \int_{\Omega} \left(2\eta\partial_x u\partial_x v + \frac{1}{2}\eta\partial_z u\partial_z v \right), d\Omega$$

where using Cacuchy-Swartz,

$$|a(u,v)| \leq 2\eta||u_x|| \ ||v_x|| + \frac{\eta}{2}||u_z|| \ ||v_z|| \leq 2\eta(||\nabla u|| \ ||\nabla v||)$$

which gives the continuity constant: $M = 2\eta$.

The Coercivity constant:α, similarly we factor the smallest cofficient, and get, $\alpha = \frac{\eta}{2}$

When applied to our Equation 9, the best approximation result for Equation 7 will be,

$$2\eta||(u-u_h)||_a + \frac{1}{2}\eta||(u-u_h)||_a \leq 4\eta\ ||u-v_h||_a \tag{13}$$

Using the association of energy norm with L^2 projection, $||(u-u_h)||_a = ||\partial_x(u-u_h)||_{L^2(\Omega)}$,

$$2\eta||\partial_x(u-u_h)||_{L^2(\Omega)} + \frac{1}{2}\eta||\partial_z(u-u_h)||_{L^2(\Omega)} \leq 4\eta\ ||\nabla(u-v_h)||_{L^2(\Omega)} \tag{14}$$

7.Priori estimate for Equation 1

For priori estimate, constructing energy norm for our Equation 7 & Equation 13 for constant viscosity,

$$||w||_a^2 = 2\eta||w_x||_a^2 + \frac{\eta}{2}||w_z||_a^2$$

which can be written as,

$$||w||_a^2 \leq 2\eta(||w_x||_a^2 + ||w_z||_a^2) = 2\eta||\nabla w||_a^2$$

Therefore, $||w||_a \leq \sqrt{2\eta}||\nabla w||_a$

for priori estimate we assume a finite element interpolant, $v_h = I_h u$,

$$||u-u_h||_a \leq\ 4C||u-I_h u||_a$$

Applying to interpolation error,

$$||u-u_h||_a \leq\ Ch||u||_{H^2}$$

square,

$$2\eta||\partial_x(u-u_h)||^2 + \frac{1}{2}\eta||\partial_z(u-u_h)||^2 \leq\ 4Ch^2||u||_{H^2}^2$$

or,eps= fem.Constant(domain,1e-15)

$$2\eta||\partial_x e||^2 + \frac{1}{2}\eta||\partial_z e||^2 \leq\ 4Ch^2||u||_{H^2}^2$$

Table 2: Convergence in the Picard iteration method with the range of epsilon value

Iteration #	1e-25	1e-20	1e-15	1e-10	1e-05	1e-01	1
0	5.29e-06	6.67e-06	2.79e-04	1.30e-02	6.05e-01	1.30e+01	2.81e+01
1	NA	NA	3.57e-05	4.22e-05	4.24e-05	4.24e-05	4.24e-05
2	NA	NA	1.01e-05	3.73e-07	8.16e-09	3.79e-10	1.76e-10
3	NA	NA	3.30e-06	NA	NA	NA	NA
4	NA	NA	NA	NA	NA	NA	NA

8. Solving the non-linearity

Now, instead of constant viscosity, we are moving to velocity dependent as described by Equation 5, with Rate factor(A) $= 3.17 \times 10^{-24}\mathrm{Pa}^{-3}\mathrm{s}^{-1}$ as mentioned in Table 1.

$$\eta = A^{-1/3}\left((\partial_x u_x)^2 + \frac{1}{4}(\partial_z u_x)^2 + \epsilon\right)^{-1/3}$$

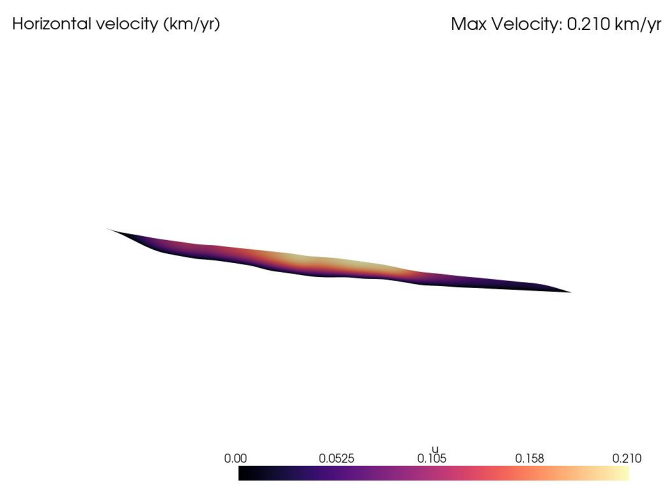

Figure 2: Glacial horizontal velocity (km/yr) solved as a non-linear PDE via Picard iteration. Parameters: $\rho = 917$, $A = 3.17 \times 10^{-24}$.

Figure 2 as compared with figure 6 of reported paper, shows strong resemblence with the maximum velocity in the central domain of the glacier.

9. ϵ and convergence

For $tolerance = 1e-5$, convergence is achieved in 3^{rd} iteration for most cases, see Table 2:

for lower ϵ values, the intial residual error is higher (28 for $\epsilon = 1$) but it converges extrememly fast to a tiny error within 3 iterations.

for higher ϵ values, the changes are slower as the residual sarts low and they barely change the solution, for $\epsilon = 1e - 15$, it takes 4 iterations to cross the threshold.

10. Result

Simulation of Swiss mountain glacier, Arolla glacier using the FO model is carried out with and without the linearity in the viscosity of the ice. The glacier is modeled with Equation 1, to determine horizontal velocity with boundary condition in Equation 4, collectively called as the FO model to study the slowly deforming glacier is it is moving down the mountain. The FO model is solved using the FEM, which is represented by Equation 7,Equation 8 along with the boundary condition, eps= fem.Constant(domain,1e-15) $>$ 1. Linear Model: When a constant viscosity is assumed, $\eta = 1.0^{15} Pa\ s$ the maximum horizontal velocity of the modeled glacier is calculated to be 1.246 m/yr(Figure 1) around the central region of glacier. $>$ 2. Non-Linear Model: The Viscosity is velocity dependent, as represented in Equation 5. When considered to model horizontal velocity of the glacier, the model becomes non-linear where the Picard Iterations methods is used. The Picard Iteration method considered the linear solution as the starting solution and within 3-4 iterations provided the solution within the tolerance limit of $1e - 5$.

The non-linear solution suggest a maximum of horizontal velocity of 0.210 km/yr as seen in Figure 2. The velocity strongly resembles the reported glacial horizontal velocity.

11. Script used

```
from mpi4py import MPI
import numpy as np
from dolfinx import fem, default_scalar_type, plot, mesh
import ufl
from dolfinx.fem.petsc import LinearProblem
import pyvista
from helpfunctions import *

# Glacial Mesh
domain = loadmesh("arolla.xdmf")

# Constants
seconds_per_year = 31556926.0
rho = fem.Constant(domain, 917.0)
```

```
g = fem.Constant(domain, 9.81)
rate_f= fem.Constant(domain, 3.17e-24)#converted to SI units, A: rate factor
eps= fem.Constant(domain,1e-15)
eta_c= fem.Constant(domain, 1e15)

# Finite element function space of Langrange polynomials of degree 1
V = fem.functionspace(domain, ("Lagrange", 2))

# Mapping cells to facets
tdim = domain.topology.dim
fdim = tdim - 1
domain.topology.create_connectivity(fdim, tdim)
boundaries = mark_bed_surface(domain)

bed_facets = boundaries.find(2)
bed_dofs = fem.locate_dofs_topological(V, fdim, bed_facets)


# Mapping BC: BED(1): Dirichlets homogeneous
uD = fem.Function(V)
uD.interpolate(lambda x: np.zeros_like(x[0]))
bc = fem.dirichletbc(uD, bed_dofs)

# Trial and Test fucntions
u = ufl.TrialFunction(V)
v = ufl.TestFunction(V)

# Gettting the height
h_surf = get_h(V, boundaries, surface_id=1)
dh_dx= ufl.grad(h_surf)[0]
# problem in weak form
a = ufl.dot(2*eta_c* ufl.Dx(u, 0), ufl.Dx(v,0) )* ufl.dx+ ufl.dot(0.5*eta_c*ufl.Dx(u, 1), uf
L = -rho*g*dh_dx* v * ufl.dx

# problem: Assembly
problem = LinearProblem(
    a,
    L,
    bcs=[bc],
    petsc_options={"ksp_type": "preonly", "pc_type": "lu"},
    petsc_options_prefix="Poisson",
)
```

```
#Solve the problem
uh_s = problem.solve()
uh = uh_s.x.array * seconds_per_year

pyvista.set_jupyter_backend('static')

topology, cell_types, geometry = plot.vtk_mesh(V)
grid = pyvista.UnstructuredGrid(topology, cell_types, geometry)
plotter = pyvista.Plotter()
plotter.add_mesh(grid, show_edges=True)
plotter.view_xy()
plotter.screenshot("glacier_mesh.jpg")
plotter.show()
plotter.close()

#Visualize: the solution

grid.point_data["u"] = uh.real
grid.set_active_scalars("u")

plotter = pyvista.Plotter()
plotter.add_mesh(u_grid, show_edges=False, cmap="magma")
plotter.add_text("Horizontal velocity (m/year)", font_size=12)
label = f"Max Velocity: {uh.max():.3f} m/yr"
plotter.add_text(label, position='upper_right', font_size=12, color='black')
plotter.view_xy()
plotter.screenshot("Velocity.jpg")
plotter.show()
plotter.close()


# Using Picard method.

#viscocity
def get_eta(u):
    return (rate_f)**(-1/3)*((ufl.Dx(u, 0))**2 + 0.25*(ufl.Dx(u, 1))**2 + eps)**(-1/3)


uh_k = fem.Function(V)
uh_k.x.array[:] = uh_s.x.array[:]
max_iter = 25
tolerance = 1e-4
```

```
for i in range(max_iter):
    eta_nl= get_eta(uh_k)
    a_nl = ufl.dot(2*eta_nl* ufl.Dx(u, 0), ufl.Dx(v,0) )* ufl.dx+ ufl.dot(0.5*eta_nl*ufl.Dx(u

    problem = LinearProblem(
    a_nl,
    L,
    bcs=[bc],
    petsc_options={"ksp_type": "preonly", "pc_type": "lu"},
    petsc_options_prefix="Poisson",
    )

    uh_p= problem.solve()


    error = np.linalg.norm(uh_p.x.array - uh_k.x.array)
    print(f"Iteration {i}: Residual Error = {error:.2e}")

    # Update the previous solution with the new one
    uh_k.x.array[:] = uh_p.x.array[:]

    if error < tolerance:
        print("Convergence achieved!")
        break

#Velocity KM/year
uh_nl = uh_p.x.array * seconds_per_year/1000
Velocity_nl#plot
pyvista.set_jupyter_backend('static')

topology, cell_types, geometry = plot.vtk_mesh(V)
grid = pyvista.UnstructuredGrid(topology, cell_types, geometry)
plotter = pyvista.Plotter()
plotter.add_mesh(grid, show_edges=True)
plotter.view_xy()
plotter.screenshot("glacier_mesh.jpg")
plotter.show()
plotter.close()

#Visualize: the solution
u_topology, u_cell_types, u_geometry = plot.vtk_mesh(V)
u_grid = pyvista.UnstructuredGrid(u_topology, u_cell_types, u_geometry)
```

```
u_grid.point_data["u"] = uh_nl.real
u_grid.set_active_scalars("u")

plotter_v = pyvista.Plotter()
plotter_v.add_mesh(u_grid, show_edges=False, cmap="magma")
plotter_v.add_text("Horizontal velocity (km/yr)", font_size=12)
label = f"Max Velocity: {uh_nl.max():.3f} km/yr"
plotter_v.add_text(label, position='upper_right', font_size=12, color='black')
plotter_v.view_xy()
plotter_v.screenshot("Velocity_nl.jpg")
plotter_v.show()
plotter_v.close()
```